

Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Computer Science and Engineering
B.Tech II Semester-III
CS231 – Object Oriented Programming Practical Assignment
Lab Assignment - 11

Q1. Username Generator

Write a C++ program that takes a person's full name as input (e.g., Sameer Joshi) and generates a username in the following format:

- Convert all characters to lowercase.
- The username should be: first three letters of the first name + last three letters of the last name.

Eg: Input : Sameer Joshi

Output : Samshi

Q2. Password Strength Checker

Create a C++ program that checks the strength of a given password. A password is strong if it:

- Has at least 8 characters.
- Contains at least one uppercase letter, one lowercase letter, one digit, and one special character.

Q3: Word Frequency Counter

Write a C++ program that counts how many times a particular word appears in a sentence entered by the user.

Q4: Student Information System

Write a C++ program using virtual base classes to avoid ambiguity in multiple inheritance. Create a class Student to store student name and roll number. Create a class Test (virtually derived from Student) to store marks in 2 subjects. Create a class Sports (virtually derived from Student) to store sports marks. Derive a class Result from both Test and Sports to calculate and display the total marks.

Eg: Name: Riya

Roll No: 12

Marks in Subject 1: 80

Marks in Subject 2: 85

Sports Marks: 10

Q5: Employee Salary Calculation

Write a C++ program to demonstrate function overriding and ways to access overridden functions. Create a base class Employee with a function showDetails() that displays the employee's basic information. Create a derived class Manager that overrides showDetails() to include additional info such as department and bonus.

Use both Manager::showDetails() and Employee::showDetails() in the same object to show how the base class function can still be accessed.

Hint: Use Employee::showDetails() syntax or scope resolution operator.

Eg: Base Details: Name = Raj, Salary = 30000

Derived Details: Department = HR, Bonus = 5000

Q6. Write a C++ program to accept a paragraph from the user and perform the following operations using in-built string functions:

- Convert the paragraph into lowercase and count the total number of words.
- Replace all occurrences of the word “C++” with “Programming”.
- Find the position of the first and last occurrence of the word “class”.
- Reverse each word in the paragraph individually (without reversing the sentence order).

Q7. Create a C++ program to demonstrate the Diamond Problem and how it can be solved using a virtual base class.

- Create a base class Person with name and age.
- Derive two classes Employee and Student from Person.
- Derive a class WorkingStudent from both Employee and Student.
- Use virtual inheritance to ensure there is only one copy of Person.
- Display all details of a WorkingStudent object and verify that ambiguity is avoided.

Q8. Design an abstract class Shape with a pure virtual function area() and a non-virtual function displayType().

- Derive classes Circle, Rectangle, and Triangle that implement the area() function.
- Demonstrate function overriding by redefining displayType() in Rectangle.
- Show the different possible ways to call the overridden displayType() (using scope resolution, base pointer, etc.).

Q9. Create a C++ program to show how containership differs from inheritance.

- Define a class Address with data members like city, state, and pincode.
- Define a class Student that contains an Address object as its data member (containership).
- Define another class GraduateStudent that inherits from Student.
- Input and display the complete details of a GraduateStudent using both concepts.
- In comments, explain how containership differs from inheritance in this example.

Q10. Write a C++ program to demonstrate the use of a virtual base class in resolving the diamond problem in multiple inheritance.

- Create a base class Person with a member variable name (string) and a method display() to print the name.
- Derive two classes Student and Employee from Person, each adding their own member variables (e.g., studentID for Student and employeeID for Employee) and overriding the display() method to include their specific details.
- Create a class Manager that inherits from both Student and Employee using virtual inheritance for the Person base class.
- In the main() function, create an object of Manager, initialize all members, and call display() to show how virtual base class avoids ambiguity in accessing name.

Q11. Implement a C++ program using an abstract class to model shapes.

- Define an abstract class Shape with a pure virtual function calculateArea() that returns a double, and a non-virtual function display() that prints a message like "This is a shape."
- Derive two concrete classes: Circle (with radius as a member) and Rectangle (with length and width as members). Implement calculateArea() for each (use 3.14 for pi in Circle).
- In the main() function, create pointers of type Shape* pointing to objects of Circle and Rectangle. Call calculateArea() and display() through these pointers.

Q12. Create a C++ program to illustrate function overriding and different ways to access the overridden base class function.

- Define a base class `Animal` with a virtual function `makeSound()` that prints "Generic animal sound."
- Derive a class `Dog` from `Animal` and override `makeSound()` to print "Bark!"
- In the `main()` function:
 - Create a `Dog` object and call `makeSound()` directly (shows overriding).
 - Use scope resolution operator (`Animal::makeSound()`) to access the base version from the `Dog` object.
 - Use a base class pointer (`Animal*`) pointing to a `Dog` object and call `makeSound()` (demonstrates polymorphism).
 - Use a base class reference to achieve the same polymorphic call.